Lab 01. Elements of object-oriented programming in the C/C++ language

A. Classes, objects, members, access specifiers.

The basics of OOP are mainly concerned with the definition of object classes, inheritance and polymorphism.

C++ classes are new types of data, which contain methods and variables, a kind of "molds" that are used to define the methods and variables of a specific object, "molded" after the model of the class.

The process of creating an object is called instantiation, in which case for each variable some concrete values will be specified (explicit or implicit).

The methods of a class can be defined inside it (inline methods) or outside, in which case it is necessary to indicate the class by the resolution or scope operator (::).

The use of so-called access (visibility) specifiers: public, private, protected serves to control the access to the variables and methods that are members of a class.

If no access specifier is mentioned, the compiler defaults to all variables and methods in the class as private.

The meaning of these specifiers in C++ is:

public: the following variables and methods are visible (accessible) from any area of the program;

private: The following variables and methods can only be accessed by class members;

protected: The following variables and methods can be accessed within the current class or in the classes derived from it.

It is usually preferred that the variables of a certain class be private, for a better control of them (they can only be modified by means of some methods in the class) and therefore for their protection.

The public methods and variables give us the way of interfacing the class and together with the private and protected ones they give us the way of implementing the class.

Public methods that aim to modify private or protected variables are called mutator/setter methods.

Public methods that aim to return private or protected variables are called accessor/getter methods.

Constructor methods are called at the time of instantiation of objects, they have not specified any kind of return type, they have the same name as that of the class to which they belong. The role of the constructors is to initialize the variables in that class and to allocate memory space corresponding to the instantiated object and the variables used within the class.

The class destructor is a method that, analogous to the constructor, can be recognized by the fact that it has the same name as the class to which it belongs, is preceded by the ~ character and has the role of freeing up the memory space allocated for the instantiated object and the variables used within the class.

B. Constructors. Destructors.

Constructors can be:

- Implicit
- Explicit
    - empty (without any argument)
    - parameterized; they usually initialize certain variables in the class and/or performs certain operations according to the variables received as arguments.

A special type of constructor is the copy constructor, whose purpose is to create a copy of an object. Copy constructors are required only if that class defines pointers that require dynamic allocation to reserve space.

Conversion constructors can be used to create instances from a predefined type or from another object.

The destructor can be:

explicit: if certain operations are desired to be performed when an object is destroyed

implicit: a default destructor is called by the compiler.

The *this* pointer points to the members of the current instance of the class (it can only be called from within the class). It is useful when inside a function of a class it is necessary to distinguish between variables belonging to different objects. The pointer *this* always indicates the current object.

To create arrays of objects we need at least one empty explicit constructor. If a parameterized constructor is used, each object in the array can be initialized by indicating an initialization list.

C. Inheritance.

Inheritance is a principle of object programming that recommends the creation of abstract models (base classes), which are later concretized (derived classes) depending on the specific problem to be solved.

This leads to the creation of a hierarchy of classes and implicitly to the reuse of the code. For many problems the solution resides only in customizing an existing solution.

The main idea within inheritance is that any *class derived* from a *base class* "inherits" all of the latter's allowed attributes and methods.

In the process of inheritance, we can restrict access to the components of the base class or modify the visibility specifiers of the class members, from the derived class's point of view:

Base class	**Inheritance**		
	private	public	protected
private member	inaccesibil	inaccesibil	inaccesibil
public member	private	public	protected
protected member	private	protected	protected

Simple inheritance is made according to the model:

```
class Nume_clasa_derivata : [specificatori_de_acces] Nume_clasa_baza
{
    //corp clasa
}
```

In the case *of multiple inheritance* , a class can inherit two or more base classes. The general syntax for specifying this type of inheritance is as follows:

```
    class Nume_clasa_derivata : modificator_de_acces1 Nume_clasa1_baza,
modificator_de_acces2 Nume_clasa2_baza [, modificator_de_acces
Nume_clasaN_baza]{...};
```

If a class is derived from more than one base class, their constructors are called in the order of derivation, and destructors are called in the reverse order of derivation.

EXAMPLES

Ex.1

```
// Rectangle.h - clasa Rectangle
class Rectangle {
    // membri private
    int height;
    int width;
public:
    // membri publici
    Rectangle(int h = 10, int w = 10); // constructor explicit cu val. implicite
    int det_area(void);
    void setHeight(int);
    void setWidth(int);
    ~Rectangle(void); // destructor explicit
};
```

```
Rectangle::Rectangle(int h, int w)     // definire constructor explicit
{
    if ((h == 10) && (w == 10))
          cout << "\nApel constructor (cu valori implicite)...";
    else
          cout << "\nApel constructor (cu valori oarecare)...";
    height = h;
    width = w;
}
Rectangle::~Rectangle(void)    // destructor
{
    cout << "\nApel destructor...";
    height = 0;
    width = 0;
}

int Rectangle::det_area(void) {
    return height * width;
}

void Rectangle::setHeight(int init_height)
{
    height = init_height;
}
void Rectangle::setWidth(int init_width)
{
    width = init_width;
}

//main()
#include <iostream>
using namespace std;
#include "Rectangle.h"

int main() {
    // obiecte de tipul Rectangle
    Rectangle rect, square;//inst.cu val implicite
    cout << "\nObiecte(acces cu operatorul .): \n";
    cout << "\tValori implicite: \n";
    cout << "\t\tAria dreptunghiului: " << rect.det_area() << endl;
    cout << "\t\tAria patratului: " << square.det_area() << endl;
    cout << "\tValori oarecare: \n";
    rect.setHeight(12);
    rect.setWidth(8);
    cout << "\t\tAria dreptunghiului: " << rect.det_area() << endl;
    square.setHeight(8);
    square.setWidth(8);
    cout << "\t\tAria patratului: " << square.det_area();
    cout << "\n.................................\n\n";
    // pointer la tipul Rectangle
    Rectangle *point;
    cout << "\nPointeri la obiecte (acces cu operatorul ->):";
    point = new Rectangle;//inst. cu val. implicite
    cout << "\n\tValori implicite: \n";
    cout << "\t\tAria dreptunghi(patrat)ului:" << point->det_area() << endl;
    cout << "\tValori oarecare: \n";
    point->setHeight(12);
    point->setWidth(12);
    cout << "\t\tAria (dreptunghi)patratului: " << point->det_area() << endl;
    delete point;
    cout << "\n.................................\n\n";
    cin.get();
}//main
```

Ex.2

```
#include <stdio.h>//e recomandat a folosi mecanismul de I/E din C++
#include <conio.h>

class Point {
        int x, y;
public :
        Point(int a=0, int b=0) {
x=a; y=b;
    }
        void setX(int a){
          x=a;
    }
        int getX(){
          return x;
    }
        void setY(int b){
          y=b;
    }
        int getY(){
          return y;
    }
};

int main(void){
Point p1;               // echivalent cu Point p1(0,0);
    printf("x = %d\t y = %d\n", p1.getX(), p1.getY());//recomandat I/E C++
    Point p2(10);           // echivalent cu Point p2(10,0);
    Point p3(10,10);
    p1.setX(20);
    p1.setY(30);
    //afisati noile valori ale atributelor x si y ale punctului p1
    _getch();
}
```

Ex.3

```
#include <iostream>
using namespace std;
#include <math.h>

class Complex{
    double re, im;
public:
    Complex(double x=0.0, double y=0.0) {
          re = x;
          im = y;}
    double modul(){
          return sqrt(re*re + im*im);}
    double faza();

    void setRe(double real){
          re=real;}
    double getRe(){
          return re;}
    void setIm(double imaginar){
          im=imaginar;}
    double getIm(){
          return im;}
```

```
    void ad_complex(Complex b){//recomandat a returna un obiect
          re += b.re;
          im +=b.im;
    }

    void sc_complex(Complex b); //recomandat a returna un obiect

};//class

int main( ){
    Complex obiect_1,obiect_2;
    double aux;
    cout <<"\nPartea reala a obiectului 1 este \t";
    cin >> aux;
    obiect_1.setRe(aux);
    cout <<"Partea imaginara a obiectului 1 este \t";
    cin >> aux;
    obiect_1.setIm(aux);
    //completati citirea valorilor reale si imaginare pentru obiectul 2 de tip
complex
    //....
    obiect_1.ad_complex(obiect_2);//adunarea  celor  doua  numere  se  face  in
obiectul 1-curent
    cout << "Partea reala a obiectului 1 este \t\n" <<obiect_1.getRe();
    cout << "Partea imaginara a obiectului 1 este \t \n" <<obiect_1.getIm();
                //apelati metoda de scadere dupa definirea ei
    cin.ignore();
    cin.get();
}
```

Ex. 4

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
using namespace std;

class Test1{
private:
    int x;
public:
    char sir[20];
    Test1(){
          cout<<"\nApel constructor explicit vid.";
    }

    //metoda care reinitializeaza variabilele din clasa
    void set_x_sir(int new_x, char new_sir[]){
          x = new_x;
          strcpy(sir, new_sir);
    }

    //metoda care returneaza valoarea variabilei private x
    int getX(){
          return x;
    }
};

int main( ){
    Test1 ob1; //instantiere
    int a;
    char s[20];
```

```
      cout<<"\nIntroduceti valoarea variabilei de tip \"int\" din clasa: ";
      cin>>a;

      cout<<"\nIntroduceti valoarea variabilei de tip \"char array\" din clasa:
";
      cin>>s;

      //apelul metodei mutator
      ob1. set_x_sir(a, s);

      //afisarea valorilor atributelor clasei

      //accesarea membrului privat prin intermediul metodei accesor
   cout<<"\nValoarea de tip \"int\" stocata in clasa si returnata de metoda accesor
este: " <<ob1.getX()<<endl;

      //accesarea directa a membrului public
      cout<<"\nValoarea de tip \"char array\" stocata in clasa si accesata direct
este: " <<ob1.sir<<endl;
      cin.ignore();
      cin.get();
   }
```

Ex. 5

```
   #include <iostream>
   using namespace std;

   class Test2{
       int x;

       Test2(){
             cout<<"\nApel constructor explicit vid.";
       }
   };

   int main( ){
       //Test2 ob1; //instantiere imposibila, constructorul este privat!
       int a;

       cout<<"\nIntroduceti valoarea variabilei de tip \"int\" din clasa: ";
       cin>>a;

       //incercare (imposibila) de initializare a variabilei din clasa
       //ob1.x = a;
   cin.ignore();
       cin.get();
   }
```

Ex. 6

```
   #include <iostream>
   using namespace std;

   const int Max1=10;
   const int Max2=10;

   class Matrix{
       //atribute
       int matrix[Max1][Max2], dim1, dim2;
       //declararea metodei de afisare a unui element
       int returnElement(int row, int column);
```

```
  public:
  //constructor explicit cu parametri - se prefera o metoda diferita a citi
valorile
      Matrix(int dim1, int dim2){
            //variabile locale
            int i, j;
            this->dim1=dim1;
            this->dim2=dim2;
            cout<<"\nIntroduceti elementele matricii: ";
            for(i=0; i<dim1; i++){
                  for(j=0; j<dim2; j++){
                        cout<<"\nmatrix["<<i<<"]["<<j<<"] = ";
                        cin>>matrix[i][j];
                  }
            }
      }

      //metoda de afisare a matricii din clasa; implementare in cadrul clasei
      void displayMatrix(){
            //variabile locale
            int i, j;

            cout<<"\nElementele matricii: ";
            for(i=0; i<dim1; i++){
                  cout<<"\n";
                  for(j=0; j<dim2; j++){
                  //apelul metodei private care returneaza valoarea unui element
din matrice
                        cout<<returnElement(i, j)<<" ";
                  }
            }
            cout<<endl;
      }
      //declararea metodei de afisare a elementelor unei coloane
      void displayColumn(int col);
  };

  //implementarea externa a metodelor (publice sau private) declarate in clasa

  void Matrix::displayColumn(int col){
      if(col<0||col >=dim2){
            cout<<"\nColoana cu numarul "<<col<<" nu exista in matricea din
clasa!\n";
      }
      else{
            cout<<"\nElementele coloanei "<<col<<": ";
            for(int i=0; i<dim1; i++){
                  cout<<returnElement(i, col)<<" ";
            }
      }
  }

  int Matrix::returnElement(int row, int column){
      return matrix[row][column];
  }


  int main( ){
      int dim1,dim2;
            cout<<"\nIntroduceti dimensiunile 1 si 2 ale matricii: (<=10):\n";
            cin>>dim1;
            cin>>dim2;
      Matrix m1(dim1,dim2); //instantiere cu citire valori
```

```
    m1.displayMatrix();
    int c;
  cout<<"\nIntroduceti un numar de coloana ale carei elemente vor fi afisate:
(<dim2) ";
    cin>>c;
    m1.displayColumn(c);
  //incercare (imposibila) de a accesa direct un membru privat al clasei
    //m1.returnElement(0, 0);

    cin.ignore();
    cin.get();
    }
```

Ex. 7

```
  #include <iostream>
  using namespace std;

  class Test4{
    //toti membri clasei sunt publici
  public:
    float f;

    Test4(){
          cout<<"\nApel constructor explicit vid.";
          //initializarea variabilei din clasa cu o valoare implicita
          f = 0;
    }

    float get_f(){//f nu e private, nu e necesara metoda
          return f;
    }
  };



  int main( ){
  //----------------------------------------------
    Test4 ob1; //instantiere (creare de obiect)
    float a;

    //afisarea atributului f din clasa, prin acces direct
    cout<<"\nValoarea atributului de tip \"float\" din clasa: "<<ob1.f;

    cout<<"\nIntroduceti o valoare de tip \"float\": ";
    cin>>a;

    //modificarea variabilei f din clasa in mod direct
    ob1.f = a;

    //afisarea atributului f din clasa, prin acces direct
    cout<<"\nValoarea variabilei de tip \"float\" din clasa: "<<ob1.f;

  //----------------------------------------------
    //declararea unui pointer la un obiect din clasa
    Test4 *Pob;
    //definirea pointerului
    Pob = new Test4;

    cout<<"\ nIntroduceti o valoare de tip \"float\": ";
    cin>>a;
```

```
    //modificarea atributului din clasa folosind un pointer
    Pob->f = a;

    //afisarea atributului f din clasa, prin intermediul metodei de tip get
    cout<<"\nValoarea atributului f de tip \"float\" din clasa: "<<Pob-
>get_f();

//---------------------------------------------
    //declararea si initializarea unei referinte la un obiect din clasa
    Test4 &Rob1 = ob1;

    //afisarea atributului din clasa, prin intermediul metodei get folosind
referinta
    cout<<"\nValoarea atributului f de tip \"float\" din clasa: "<<Rob1.get_f();
cin.ignore();
    cin.get();

}
```

Ex. 8

```
class Baza {
protected:
    int i, j;
public:
    void setI(int a) {
          i = a;
    }
    void setJ(int b) {
          j = b;
    }
    void afiseaza() {
          cout << "\n Valori atribute: i, j: ";
          cout << i << ", " << j << endl;
    }
};//Baza class
class Derivata : public Baza {
public:
    int inmulteste() {
          return (i * j);   // corect, i si j raman protected
    }
};//Derivata class
int main( ) {
    Derivata obiect_derivat;
    obiect_derivat.afiseaza(); //val. nedefinite
    //obiect_derivat.i = 5; // gresit, i  este protected
    obiect_derivat.setI(12); //din Baza
    obiect_derivat.setJ(17); //din Baza
    obiect_derivat.afiseaza(); //din Baza
cout << "\n Produsul este: " << obiect_derivat.inmulteste();//din Derivat
    cin.get();
    return 0;
} //main
```

Ex. 9

```
#include <iostream>
using namespace std;
class Baza{     int x;
    protected:
          int y;
    public:
```

```
            int z;
        Baza(int x=0, int y=0 ) {
            this->x=x;
            this->y=y;
            }//Baza
        void arata( ) {
    cout << "\n --------Clasa de baza------";
    cout << "\n Valoarea variabilei private x: "<<x<<endl;
    cout << "\n Valoarea variabilei protected y: "<<y<<endl;
            }//arata
        };//Baza class
    class Derivata : protected Baza {
    public:
        void do_this(void) {
            cout << "\n --------Clasa derivata------";
            //cout << "\n Valoarea variabilei private x: " << x << endl;//ramasa
private
            cout << "\n Valoarea variabilei protected y: " << y << endl;//
protected
            cout << "\n Valoarea variabilei  z: " << z << endl;//devenita protected
            y = 7; cout << "y= " << y << endl;//corect, y ramane protected
            z = 15; cout << "z= " << z << endl;// corect, z devine protected
                }//do_this
        };//Derivata class
        int main( ){
            int x, y;
            cout << "\n x= "; cin >> x;
            cout << "\n y= "; cin >> y;
            Baza ob1(x, y);
            Derivata ob2;
            ob1.arata();//public in Baza
            //ob2.arata(); protected in Derivata
            ob2.do_this();
            cin.ignore();
            cin.get();
            return 0;
        }//main
```

Ex. 10

```
#include <iostream>
using namespace std;

class Baza
{
protected:  int a, b;
public:
    void setA(int a) { this->a = a; }
    void setB(int b) { this->b = b; }
    Baza() { a = 1, b = 1;}
    int aduna() {
        return a + b;
    }
    int scade() {
        return a - b;
    }
    void afis_baza() {
        cout <<"\nAfis din baza: " << a << " " << b << '\n';
    }
};
class Derivata : private Baza
{
public:
```

```
    int inmulteste() {
          return a * b;
    }
};
int main(void)
{
    Baza obiect_baza;
    obiect_baza.afis_baza();
    obiect_baza.setA(2);
    obiect_baza.setB(3);
    obiect_baza.afis_baza();
    Derivata obiect_derivat;
cout << "\nProdusul este= " << obiect_derivat.inmulteste(); // corect val.
implicite
     //obiect_derivat.aduna();         // eroare, aduna () devine private
    cin.get();
    return 0;
}
```

Ex. 11

```
class Baza1 {
 protected:
    int x;
 public:
    void arata_x( ) { cout << "B1: valoarea lui x este: " << x <<endl;}
    void arata( )  { cout << " Din Baza1: x = " << x <<endl;}
    };//Baza1 class
class Baza2 {
protected:
    int y;
public:
    void arata_y( ) { cout << "B2: valoarea lui y este: " << y <<endl;}
    void arata( ) { cout << " Din Baza2: y = " << y <<endl;}
    };//Baza2 class
class Derivata : public Baza1, public Baza2 {
public:
    void setX(int i) { x = i; }
    void setY(int j) { y = j; }
          void arata() {
                cout << "\nDin Derivata: \n";
                Baza1::arata();
                Baza2::arata();
          }
}; //Derivata class
int main(void) {
    Derivata obiect_derivat;
    obiect_derivat.setX(100); obiect_derivat.setY(200);
    obiect_derivat.arata_x();//from Baza1
    obiect_derivat.arata_y();//from Baza2
    obiect_derivat.arata();//from Derivata class
    cin.get();
    return 0;
}//main
```

Ex. 12

```
#include <iostream>
using namespace std;
// clasa de baza
class Pozitie {
protected:
    int x, y;
```

```
public:
    Pozitie(int = 0, int = 0);
    void afisare();
    void deplasare(int, int);
};//Pozitie class
Pozitie::Pozitie(int abs, int ord) {
    x = abs; y = ord;
    cout << "Constructor \"Pozitie\", ";
    afisare();
}
void Pozitie::afisare() { cout << "coordonate: x = " << x << ", y = " << y <<
endl; }
void Pozitie::deplasare(int dx, int dy) {
    x += dx; y += dy;
}

// clasa derivata
class Punct : public Pozitie {
    int vizibil;
    char culoare;
public:
    Punct(int = 0, int = 0, char = 'A');
    void arata() {
          vizibil = 1;
    }
    void ascunde() {
          vizibil = 0;
    }
    void coloreaza(char c) {
          culoare = c;
    }
    void deplasare(int, int);
    void afisare();
};//Punct class
  // constructor
Punct::Punct(int abs, int ord, char c) :Pozitie(abs, ord)
{
    vizibil = 0;
    culoare = c;
    cout << "Constructor \"Punct\", ";
    afisare();
}
// definire alta metoda de afisare in clasa derivata
void Punct::afisare() {
    cout << "Pozitie: x = " << x << ", y = " << y;
    cout << ", culoare: " << culoare;
    if (vizibil)
          cout << ", vizibil \n";
    else
          cout << ", invizibil \n";
}
// definire alta metoda de deplasare in clasa derivata
void Punct::deplasare(int dx, int dy)
{
    if (vizibil) {
          cout << "Deplasare: ";
          afisare();
    }
    x += dx;
    y += dy;
    if (vizibil) {
          cout << "la coordonatele: ";
          Pozitie::afisare();
    }
```

```
}
// program ce utlizeaza clasa derivata
int main(void) {
    Punct p0(1, 1, 'V');
    Punct p1(p0); //copy-constructor implicit
    p1.arata();
    p1.deplasare(10, 10);
    cin.get();
    return 0;
}//main
```